Engineering Specification & Architectural Blueprint

Self-Hosted Agentic Marketing & Social Media Automation Platform (Ollama + LangGraph + Temporal.io + gVisor + PostgreSQL/pgvector)

Executive Document Scope & Architectural Summary

This implementation-ready engineering specification transforms the conceptual design for a self-hosted, Ollama-powered agentic marketing and social media automation platform into a technically validated system blueprint. Every architectural decision, external API integration contract, rate-limit ceiling, and security boundary is grounded in verified primary documentation across eight target social networks and open-source infrastructure components.

Specification Dimension	Validated Engineering Standard
Cognitive Control Plane	Python 3.12 + LangGraph (`langgraph>=1.2.12`) with `PostgresSaver` durable checkpoints and typed `interrupt()` human-in-the-loop governance gates.
Local Inference Engine	Self-hosted Ollama server (`qwen3:32b` / `qwen3:14b` for orchestration, `qwen3:8b` for worker extraction, `gemma4` for vision QA, `bge-m3` for embeddings) with strict separation between tool-calling turns and JSON Schema `format` turns.
Worker Sandbox Plane	Ephemeral Docker containers executing under the gVisor (`runsc`) application kernel on an `internal: true` bridge network, routing all external tool and web access through an HMAC-authenticated MCP Tool Gateway and Envoy forward proxy.
Durable Execution & Scheduling	Temporal.io workflows managing multi-week content calendars, OAuth 2.0 token rotation, resumable/chunked binary media uploads, rate-limit backoff, and 6h/24h telemetry polling.
Target Platform Adapters	Direct API contracts for LinkedIn (`/rest/posts`), YouTube Data API v3 (`videos.insert`), Instagram Graph API (`v26.0`), Threads API (`v1.0`), Facebook Pages API (`v26.0`), X API v2 (`/2/tweets`), Pinterest API v5 (`/v5/pins`), and Google Business Profile API v4 (`localPosts`).
Unified Storage & RAG	PostgreSQL 16 + `pgvector` hybrid retrieval (HNSW cosine similarity + `tsvector` full-text search with Reciprocal Rank Fusion) + MinIO S3-compatible media storage with signed HTTPS staging ingress.

Implementation Roadmap & Verification Gate Summary

Milestone	Core Engineering Deliverable & Acceptance Gate
M1: Core Vertical Slice	Docker Compose stack, AES-256-GCM Credential Vault, LangGraph + Ollama `campaign.json` compiler, HITL `interrupt()` gate, and verified publishing to LinkedIn & X.
M2: Sandboxed Workers & RAG	gVisor (`runsc`) ephemeral worker containers on an `internal: true` network, Envoy allowlist proxy, self-hosted SearXNG, and `pgvector` hybrid RAG store.
M3: Media Pipeline	Fal.ai & Replicate image/video synthesis workers, deterministic FFmpeg/Pillow normalization, `gemma4` vision QA, and 15-minute signed HTTPS staging ingress.
M4: 8-Platform Adapters	Production adapters for Instagram, Threads, Facebook Pages, YouTube Data API v3 (resumable `308` upload), Pinterest v5, and Google Business Profile v4 (`localPosts`).
M5–M6: Workspace & Analytics	Next.js 15 Unified Workspace (SSE Agent Monitor, Calendar, Media Library) + closed-loop 6h/24h telemetry ingestion, weekly Parquet backups, and cohort Z-score optimization.

Document Classification: Principal Systems & AI Architecture Specification **Verification Date:** October 31, 2026 **Target Release:** v1.0 Production Blueprint

1. Executive Assessment: Hybrid Deterministic-Agentic Architecture Delivers Self-Hosted Marketing Autonomy

Transforming the proposed self-hosted, Ollama-powered marketing and social media automation concept into a reliable production system requires separating probabilistic language model deliberation from deterministic infrastructure execution. While the concept envisions a central Master Orchestrator dynamically spawning isolated ephemeral worker agents to conduct market research, generate multi-modal campaign assets, publish across eight social networks, and execute closed-loop analytics optimization, treating every system operation as an autonomous agent loop introduces severe failure modes. Local open-weight language models running via Ollama exhibit finite context windows, non-trivial VRAM footprints, and known failure patterns when combining strict JSON schema enforcement with multi-turn function calling (Ollama Issue #8095). Simultaneously, external social media platforms enforce rigid OAuth 2.0 token lifecycles, multi-step chunked binary upload protocols, strict daily publishing caps, and mandatory compliance audits that cannot be bypassed by local software architecture alone.

The technically validated architecture recommended in this blueprint establishes a **Hybrid Control-and-Execution Plane**. A stateful agentic orchestration layer built on LangGraph and backed by a PostgreSQL Checkpointer governs high-value cognitive workflows: decomposing user intent, coordinating ephemeral research workers inside gVisor (`runsc`) sandboxes, synthesizing brand context from a pgvector hybrid knowledge base, and compiling validated `campaign.json` blueprints. Once a campaign blueprint reaches human approval via LangGraph `interrupt()` gates, execution transitions out of the LLM loop into a deterministic workflow engine powered by Temporal.io. Temporal manages long-horizon calendar scheduling, OAuth token refresh cycles, resumable media uploads, rate-limit backoff, and periodic analytics ingestion across LinkedIn, YouTube, Instagram, Threads, Facebook, X (Twitter), Pinterest, and Google Business Profile.

Architectural Dimension	Original Concept Proposal	Validated Engineering Blueprint	Primary Technical Justification
Core Orchestration	Single Ollama Master Orchestrator managing all planning, scheduling, and API retries	LangGraph for cognitive planning + Temporal.io for durable scheduling and API execution	Prevents LLM hallucination during multi-day delays, OAuth token refreshes, and chunked binary uploads (Temporal Durable Execution).
Worker Isolation	Dynamically spawned isolated worker environments with controlled tool access	Pre-warmed Docker containers running under the gVisor (`runsc`) application kernel on an `internal: true` bridge network	Eliminates host kernel syscall exposure while enforcing egress allowlists via an internal HTTP/CONNECT proxy (gVisor Security).
Local LLM Engine	Unspecified local Ollama models for all tasks	Tiered Ollama deployment (`qwen3:32b` or `qwen3:14b` for orchestration; `qwen3:8b` for worker extraction; `bge-m3` for embeddings)	Separates tool-calling turns from `format` JSON schema extraction turns to avoid empty tool-call regressions (Ollama Issue #8095).
Multi-Platform Publishing	Direct API integrations across 8 platforms without third-party SaaS schedulers	Deterministic per-platform adapter activities with mandatory public TLS media ingress (Cloudflare R2 / S3 or Reverse Tunnel to MinIO)	Instagram, Threads, and Google Business Profile cURL media from public HTTPS URLs and reject private LAN file uploads.
Analytics Optimization	Continuous autonomous Observe–Analyze–Act–Evaluate loop adjusting live campaigns	Scheduled deterministic ingestion into PostgreSQL time-series tables triggering bounded diagnostic subgraphs with human approval gates	Protects against runaway API quota exhaustion and brand-damaging autonomous edits caused by premature sample sizes.

2. Problem Definition: Eliminating SaaS Lock-In While Navigating Platform API Gatekeeping

2.1 Operational Context and Core Problem Statement

Growth teams, digital agencies, and solo brand operators face escalating subscription costs, rigid workflow templates, and data residency risks when relying on commercial SaaS marketing suites and social media schedulers. Commercial platforms impose per-seat and per-channel pricing tiers, restrict raw access to cross-platform performance telemetry, and route proprietary brand strategy through multi-tenant cloud language models. Conversely, existing self-hosted open-source schedulers such as Postiz and Mixpost focus primarily on calendar publishing and basic AI copy assistance rather than end-to-end market research, structured campaign synthesis, sandboxed competitor crawling, and closed-loop performance optimization.

The engineering problem is to design and specify a fully self-hosted, single-tenant or agency-grade marketing operating system in which local language models served by Ollama act as the cognitive control plane, while isolated worker environments and deterministic API adapters execute market intelligence gathering, external media synthesis, multi-network publishing, and telemetry-driven campaign refinement. Crucially, the architecture must confront a foundational operational reality: **eliminating third-party SaaS schedulers does not eliminate external platform gatekeeping**. Every self-hosted instance must independently register developer applications, pass platform compliance audits, maintain verified business portfolios, and respect per-project rate limits across Meta, Google, LinkedIn, X, and Pinterest.

2.2 Intended System Operators and Primary Outcomes

The system is architected for three primary operator profiles:

1. **Technical Brand Operators and Solo Founders** seeking complete data sovereignty and zero recurring SaaS seat fees while automating multi-channel content calendars.
2. **In-House Growth and Performance Marketing Engineers** requiring deep customization of retrieval-augmented brand knowledge, automated competitor tracking, and programmatic A/B creative iteration.
3. **Self-Hosted Digital Agencies** managing multiple isolated brand workspaces on dedicated on-premises or virtual private server (VPS) hardware with strict credential encryption and human approval workflows.

The expected system outcomes are:

- Automated synthesis of verifiable market, audience, and competitor intelligence into structured strategy artifacts stored in a persistent pgvector knowledge base.
- Deterministic compilation of multi-platform campaign blueprints (`campaign.json`) validated against strict Pydantic schemas and platform-specific media constraints.
- Automated orchestration of external image and video generation APIs (Fal.ai, Replicate) with local asset normalization via `FFmpeg` and `Pillow`.
- Direct, auditable publishing and scheduling across eight target social networks with cryptographic credential isolation and verified external side-effect confirmation.

- Closed-loop telemetry ingestion that evaluates post-level performance deltas against historical baselines and proposes evidence-backed schedule or creative adjustments.

2.3 System Boundaries, Exclusions, and Input Ambiguities

Because the initial concept specification (`idea.md`) omits existing repository URLs, hardware specifications, budget ceilings, and developer account statuses, all unspecified parameters are treated as unknown. In accordance with strict requirements engineering discipline, the table below documents the identified input ambiguities, the explicit engineering assumptions adopted to construct this specification, and the targeted clarification questions required from the system owner before writing production code.

Ambiguity Domain	Unspecified Detail in `idea.md`	Explicit Engineering Assumption Adopted for Specification	Required Clarification Question for System Owner
Existing Codebase	No repository URL or existing project files provided	Greenfield implementation (`v1.0`) designed as a modular monorepo; no legacy migration constraints assumed.	Do you have an existing repository or prototype codebase that must be preserved, or is this a clean-slate greenfield build?
Local GPU Hardware	Hardware VRAM, CPU cores, and RAM unspecified for local Ollama execution	Dedicated Linux host or workstation with at least 24 GB VRAM (e.g., NVIDIA RTX 3090/4090) and 64 GB system RAM to run 14B–32B quantized models with 32k context.	What exact GPU model(s), VRAM capacity, and system RAM are available on your target self-hosted server?
Public Ingress / DNS	Self-hosted network topology (NAT/home lab vs. cloud VPS with public IP) unspecified	System possesses a public HTTPS domain or Cloudflare Tunnel capable of serving OAuth 2.0 callbacks, platform webhooks, and temporary signed media URLs.	Will the host run on a home/office LAN behind NAT (requiring a tunnel/CDN bucket) or on a cloud VPS with a dedicated public IP and domain?
Platform App Approvals	Current status of Meta Business Verification, Google Cloud audits, LinkedIn Company Page admin roles, and GBP 60-day eligibility	Developer accounts start unverified; architecture includes explicit mock/sandbox harnesses and tracks platform approval gates as external dependencies.	Which of the 8 target platforms already have approved developer apps, verified business entities, or active 60-day Google Business Profiles?
Paid API Budgets	Budget for X API pay-per-use credits and external media generation APIs (Fal.ai, Replicate)	Operational budget accommodates external per-call costs ($0.01/post on X API v2; ~$0.03–$0.50 per generated image/video via Fal.ai or Replicate).	What is your monthly operational budget ceiling for external media generation APIs and X (Twitter) API pay-per-use credits?
Ad Spend Scope	Mention of "advertising campaigns" alongside organic social publishing	Phase 1–3 scope is restricted to **organic publishing, dark post creation, and analytics**; automated paid media budget allocation (Meta Ads / LinkedIn Ads bidding) is excluded from autonomous execution due to financial risk.	Should paid ad campaign management be excluded from v1 autonomous execution, or restricted strictly to read-only reporting and dark-post drafting?

3. Requirements Specification: Testable Functional, Non-Functional, and Governance Controls

3.1 Core User Workflows and Human-in-the-Loop Governance

The platform operates across four primary workflows governed by explicit human approval gates:

1. **Workflow 1: Market Research and Strategy Formulation (`WF-RES`)**: The user submits a high-level brand objective to the Master Orchestrator. The orchestrator queries the pgvector knowledge base for existing brand context, spawns isolated ephemeral research workers inside gVisor containers to crawl competitor pages and public search APIs, and synthesizes a structured Strategy Document. **Governance Gate (`HITL-01`)**: The user reviews and approves or edits the Strategy Document before chunks are committed to the persistent RAG store.
2. **Workflow 2: Campaign Blueprinting and Media Production (`WF-CMP`)**: Using the approved strategy, the Master Orchestrator generates a schema-validated `campaign.json` specifying platform-tailored copy, hashtags, publishing timestamps, and media prompts. Dedicated Media Workers invoke external generation APIs (Fal.ai / Replicate), download raw assets into the local Media Library, and execute deterministic transcoding (`FFmpeg` / `Pillow`) to verify aspect ratios, codecs, and file sizes. **Governance Gate (`HITL-02`)**: Execution pauses via LangGraph `interrupt()`; the Unified Workspace displays rendered post previews for human sign-off.
3. **Workflow 3: Multi-Platform Scheduled Publishing (`WF-PUB`)**: Upon human approval of `HITL-02`, the backend registers durable workflow schedules in Temporal.io. At each scheduled timestamp, the platform-specific publishing activity validates OAuth token freshness, stages media to a signed public HTTPS URL (when required by Meta/GBP/Pinterest), executes the platform upload and publish calls, polls asynchronous container processing statuses, and verifies the resulting live post URN/ID via a read-after-write check.
4. **Workflow 4: Closed-Loop Analytics and Optimization (`WF-OPT`)**: Scheduled Temporal cron workflows poll authorized platform analytics endpoints at 6-hour and 24-hour intervals, persisting normalized metrics into PostgreSQL time-series tables and taking weekly immutable snapshots. An analytical evaluation node compares cohort performance (engagement rate, reach, click-through rate) against rolling baselines, deploys a diagnostic worker to inspect outliers, and drafts an Optimization Proposal for the next campaign cycle (**Governance Gate `HITL-03`**).

3.2 Prioritized Functional Requirements

Requirements are prioritized using MoSCoW notation (**P0 = Must Have for MVP Vertical Slice**, **P1 = Should Have for Production v1.0**, **P2 = Could Have / Phase 2 Enhancement**) with explicit, testable acceptance criteria.

Req ID	Priority	Category	Functional Requirement Specification	Measurable Verification & Acceptance Criteria
FR-01	**P0**	Orchestration	Master Orchestrator stateful graph execution using LangGraph and Ollama with persistent checkpointing in PostgreSQL.	Graph state survives process restart (`SIGKILL`) mid-workflow and resumes from exact `thread_id` checkpoint with zero data loss.
FR-02	**P0**	Orchestration	Dynamic ephemeral worker lifecycle management via Docker Engine API using the gVisor (`runsc`) runtime and scoped tool registries.	Worker container spawns in `<3s`, executes assigned tool calls, returns structured JSON output, and is destroyed (`docker inspect` returns 404) within `5s` of task completion.
FR-03	**P0**	Governance	Mandatory human-in-the-loop pause/resume gates using LangGraph `interrupt()` and typed Pydantic `response_schema` validation before any external publishing or media spend.	Invoking a publishing node without a signed approval record in `approval_audit_log` raises a deterministic guard exception; resuming with `Command(resume=...)` transitions state cleanly.

Req ID	Priority	Category	Functional Requirement Specification	Measurable Verification & Acceptance Criteria
FR-04	**P0**	Blueprinting	Deterministic `campaign.json` generation and validation enforcing platform-specific character limits, media counts, and aspect ratios via Ollama Structured Outputs.	100% of accepted `campaign.json` payloads pass Pydantic schema validation and platform constraint rules (e.g., X `≤280` chars, Threads `≤500` chars, GBP `≤1,500` chars, Instagram JPEG-only for images).
FR-05	**P1**	Research	Sandboxed web research and competitor analysis workers routing HTTP requests exclusively through an allowlisted forward proxy and self-hosted search/crawling tools.	Worker attempts to connect directly to arbitrary IP addresses on port 80/443 outside the proxy fail with network unreachable; authorized crawl returns clean Markdown and source URLs.
FR-06	**P1**	Media	External AI image and video generation integration (Fal.ai, Replicate) paired with local `FFmpeg`/`Pillow` transcoding and centralized Media Library storage.	Generated video is automatically transcoded to H.264/AAC MP4 with `moov` atom at front, verified via `ffprobe`, hashed (`SHA-256`), and stored in MinIO with full provenance metadata.
FR-07	**P0**	Publishing	Direct OAuth 2.0 / API publishing and scheduling adapter for **LinkedIn** (`/rest/posts`, Images/Videos/Documents APIs) and **X (Twitter)** (`/2/tweets`, `/2/media/upload` chunked flow).	End-to-end test publishes a text+image post to LinkedIn and X, captures the returned `x-restli-id` / `tweet.id`, and verifies live existence via a subsequent `GET` request.
FR-08	**P1**	Publishing	Direct container-based publishing adapters for **Instagram** (`/<IG_ID>/media`, `/media_publish`), **Threads** (`/<THREADS_USER_ID>/threads`, `/threads_publish`), and **Facebook Pages** (`/<PAGE_ID>/feed`, `/photos`, `/videos`).	Adapter stages media to a public signed HTTPS URL, creates container, polls `status_code == FINISHED`, publishes container, and records the verified platform permalink.
FR-09	**P1**	Publishing	Direct publishing adapters for **YouTube Data API v3** (`videos.insert` resumable upload) and **Pinterest API v5** (`POST /v5/pins`).	Adapter completes chunked/resumable upload, handles HTTP `308 Resume Incomplete` recovery on simulated network drop, and verifies returned video/Pin ID.
FR-10	**P2**	Publishing	Direct publishing adapter for **Google Business Profile API** (`POST /v4/accounts/{accountId}/locations/{locationId}/localPosts`).	Adapter validates project quota (`>0 QPM`), strips prohibited phone numbers/emails from `summary` text to prevent `REJECTED` policy state, and confirms `state == LIVE` or `PROCESSING`.
FR-11	**P1**	Knowledge	Agentic RAG knowledge base using PostgreSQL + pgvector hybrid search (`tsvector` BM25/FTS + HNSW cosine similarity) with Reciprocal Rank Fusion (RRF).	Queries for brand tone, past campaign metrics, or audience segments return top-k chunks with source citations in `<150ms` at P95.
FR-12	**P1**	Analytics	Closed-loop analytics ingestion, weekly automated snapshots, and anomaly detection workflow triggering optimization proposals.	System ingests post impressions/engagements on schedule, writes immutable weekly Parquet/JSONL backups, and generates a cited optimization diff when engagement drops `>20%` below rolling mean.
FR-13	**P1**	Workspace	Unified web interface (Next.js) providing Analytics Dashboard, Content Calendar, Media Library, RAG Inspector, Campaign Manager, and live Agent Telemetry via SSE.	Operator can view real-time LangGraph node transitions, inspect worker tool logs, drag-and-drop calendar slots, and approve/reject `interrupt()` payloads from the browser.

3.3 Non-Functional Requirements

NFR ID	Domain	Requirement Specification	Target Metric / Verification Standard
NFR-01	**Security & Credentials**	All OAuth 2.0 access tokens, refresh tokens, and external API keys must be encrypted at rest using envelope encryption (`AES-256-GCM`) with keys derived outside the database table. Worker containers must never receive raw OAuth tokens in environment variables.	Database dump inspection confirms zero plaintext tokens; worker `env` inspection confirms zero social platform credentials inside ephemeral sandboxes.
NFR-02	**Network Isolation**	Ephemeral worker containers must run on a Docker bridge network with `internal: true` (no default gateway to the host NIC). External HTTP/HTTPS calls must traverse an authenticated Squid/Envoy proxy enforcing domain allowlists.	Automated security test inside worker container attempting raw TCP connection (`nc 1.1.1.1 53` or `curl https://example.com` bypassing proxy) fails 100% of the time.
NFR-03	**Reliability & Idempotency**	Every external publishing call must carry a deterministic idempotency key (`SHA-256(campaign_id + platform + slot_id + content_hash)`) checked against PostgreSQL before network transmission to prevent duplicate social posts during retries.	Replaying a failed Temporal activity after a network timeout post-HTTP-201 detects the committed state or idempotency lock and produces zero duplicate posts.
NFR-04	**Performance & VRAM**	Local LLM orchestration must prevent GPU out-of-memory (OOM) thrashing by serializing or bounding concurrent Ollama model loads (`OLLAMA_MAX_LOADED_MODELS=2`, `OLLAMA_NUM_PARALLEL=2`).	10 concurrent worker requests complete without CUDA/ROCm OOM errors; P95 tool-call planning latency remains `<12s` on reference 24 GB VRAM hardware.
NFR-05	**Observability & Audit**	Every LLM prompt, tool invocation, container lifecycle event, OAuth token refresh, and external API request/response status must emit structured OpenTelemetry traces and immutable audit rows.	100% of external side effects can be traced from live platform post URN back to the originating `campaign.json` version, worker trace ID, and human approval signature.
NFR-06	**Maintainability & Versioning**	Platform API adapters must isolate version headers (e.g., LinkedIn `Linkedin-Version: 202610`, Meta Graph `v26.0`) in configuration and support independent contract testing via recorded HTTP fixtures.	Upgrading a single platform API version requires zero modifications to the Master Orchestrator or `campaign.json` schema.

4. Technical Research: Evidence-Backed Evaluation of Open-Source Schedulers, Orchestrators, and Local Models

4.1 Evaluation of Existing Open-Source Self-Hosted Marketing and Scheduling Systems

Before designing custom infrastructure, mature open-source self-hosted social media platforms were evaluated against the requirements in `idea.md` using official repositories and license texts.

System / Repository	License	Core Stack	Supported Capabilities	Architectural Fit & Limitations for Agentic Platform
Postiz (`gitroomhq/postiz-app`)	GNU AGPL-3.0	TypeScript monorepo (`pnpm`), Next.js, NestJS, Prisma, PostgreSQL, Redis, **Temporal**, MCP server	Scheduling across 15+ networks (Instagram, YouTube, LinkedIn, Facebook, Pinterest, Threads, X), calendar UI, analytics, AI copilot, MCP/CLI connectors.	**High Reference Value / Strong Copyleft Constraint**: Postiz validates the use of **Temporal** for social scheduling and provides battle-tested TypeScript provider patterns. However, its core is coupled to Next.js/NestJS under AGPL-3.0 and lacks gVisor-sandboxed ephemeral research workers, local Ollama ReAct orchestration, and closed-loop RAG optimization.
Mixpost Lite (`inovector/mixpost`)	MIT License (Lite) / Commercial License (Pro & Enterprise)	PHP / Laravel, Vue.js, Inertia.js, Redis, MySQL/PostgreSQL	Self-hosted scheduling and calendar management for basic social channels in Lite; Threads, Pinterest, GBP, and advanced analytics are gated behind paid Mixpost Pro/Enterprise.	**Rejected as Foundation**: The open-source Lite edition omits several required target platforms (Threads, GBP, YouTube Shorts workflows) and relies on a monolithic PHP/Laravel queue architecture poorly suited for Python/TypeScript AI agent orchestration.
n8n (`n8n-io/n8n`)	Sustainable Use License (Fair-Code)	TypeScript, Node.js, Vue.js, SQLite/PostgreSQL	Visual node-based workflow automation with HTTP Request nodes, webhook triggers, and basic LangChain agent nodes.	**Rejected as Core Engine**: Not an all-in-one marketing workspace (no native content calendar, media library, or cross-platform analytics dashboard), and Fair-Code licensing restricts certain commercial distribution models.

Architectural Conclusion on Reuse: Rather than forking Postiz (which would bind the entire codebase to AGPL-3.0 and a monolithic NestJS structure), the engineering team should adopt Postiz's proven architectural pattern—using **Temporal.io for durable scheduling and per-platform provider interfaces**—while building a clean, modular Python/TypeScript platform tailored for local Ollama orchestration, LangGraph state machines, and gVisor worker sandboxing.

4.2 Agentic Orchestration and Durable Execution Frameworks

A critical architectural flaw in many agentic systems is conflating **cognitive decision-making** (which requires an LLM state machine) with **time-delayed durable execution** (such as waiting 14 days to publish a post or polling a video transcoding endpoint with exponential backoff).

Framework / Engine	Official Source & License	Architectural Role	Strengths & Verified Capabilities	Known Limitations & Failure Modes
LangGraph (`langgraph` + `langgraph-checkpoint-postgres`)	LangChain Docs / MIT License	Stateful cognitive orchestration, cyclic ReAct loops, dynamic worker fan-out, and HITL gates.	Native PostgresSaver persists graph state at every super-step; native `interrupt()` and `Command(resume=...)` support JSON Schema validation (`langgraph>=1.2.12`) for human approvals.	Nodes restart from the beginning upon `Command(resume=...)` (LangGraph Interrupt Rules); all pre-interrupt operations must be idempotent (`upsert`), and bare `try/except` blocks around `interrupt()` break suspension. Not designed for multi-week cron timers.
Temporal.io (`temporalio/temporal`)	Temporal Docs / MIT License	Durable workflow execution for scheduled publishing, OAuth token rotation, chunked uploads, and analytics polling.	Guarantees exact-once workflow state progression across server crashes; supports multi-week durable timers (`workflow.sleep`), per-activity retry policies, and heartbeating for multi-gigabyte video uploads (Temporal Production Guide).	Requires running the Temporal Server service alongside PostgreSQL; workflow code must be strictly deterministic (all I/O isolated inside Activities).
Celery / **BullMQ**	BSD / MIT License	Traditional Redis-backed task queues for fire-and-forget background jobs.	Lightweight setup for simple single-step jobs.	Retrying a failed multi-step publishing flow (e.g., failing at step 3 of container creation → upload → publish) either re-runs the entire task or requires custom state-machine boilerplate (Celery vs Temporal Comparison).

4.3 Local Inference Engine: Ollama Capabilities, Constraints, and Hardware Sizing

The platform mandates local Ollama models as the primary cognitive engine. Inspection of official Ollama documentation and GitHub issue trackers reveals three critical engineering constraints that directly shape the system design:

1. **Tool Calling vs. Structured Outputs Mutual Exclusivity Risk**: Ollama natively supports Structured Outputs (passing a Pydantic/Zod JSON Schema in the `format` parameter) and Tool Calling (passing function schemas in the `tools` parameter, including parallel tool calls and `think=True` deliberation traces on models such as `qwen3`). However, combining `format` (JSON schema) and `tools` in the **same** `/api/chat` call is a documented failure mode across multiple Ollama versions and wrappers (Ollama Issue #8095, LiteLLM Issue #17807), frequently returning an empty `tool_calls` array. - **Engineering Design Rule**: Every agent node must strictly separate **Tool-Calling Turns** (`tools=[...]`, `format=None`) from **Structured Extraction/Blueprinting Turns** (`tools=None`, `format=CampaignBlueprint.model_json_schema()`, `options={"temperature": 0}`).
2. **Context Window Truncation (`num_ctx`)**: By default, Ollama allocates a conservative context window (often 2,048 or 4,096 tokens depending on model defaults) unless explicitly overridden via `options={"num_ctx": 32768}` in the API payload or a custom `Modelfile`. Multi-turn ReAct loops ingesting competitor web pages or cross-platform analytics tables silently truncate system prompts and tool definitions if `num_ctx` is not explicitly set.
3. **VRAM Contention Across Ephemeral Workers**: If the Master Orchestrator and four parallel ephemeral workers simultaneously request different model weights (e.g., a 32B orchestrator model, a 14B coder model, and an 11B vision model), Ollama will continuously swap weights between system RAM and GPU VRAM, degrading throughput from ~40 tokens/sec to <2 tokens/sec. - **Engineering Design Rule**: Standardize on **one shared primary instruction/tool-calling model** kept resident in VRAM (`keep_alive=-1`), **one lightweight embedding model** (`bge-m3` or `nomic-embed-text`), and an on-demand **vision model** (`gemma4` or `qwen2.5-vl`) invoked sequentially during media QA.

Hardware Tier	GPU / VRAM Configuration	Recommended Resident Ollama Models	Max Concurrent Agent Threads (`OLLAMA_NUM_PARALLEL`)	Expected Planning & Blueprint Throughput
Tier 1: Minimum Viable Self-Hosted	Single 16 GB–24 GB GPU (e.g., RTX 4080 / RTX 3090 / RTX 4090) or Apple M-Series 32 GB Unified	Primary: `qwen3:14b-q4_K_M` (~9.5 GB VRAM @ 32k `num_ctx`) Embeddings: `bge-m3` (~1.2 GB) Vision (on-demand): `gemma4:8b`	2 parallel requests	~35–55 tok/s; full `campaign.json` synthesis in ~15–25 seconds.
Tier 2: Production Recommended	Dual 24 GB GPUs (48 GB total VRAM, e.g., 2× RTX 3090/4090) or Apple Mac Studio 64 GB+ Unified	Orchestrator: `qwen3:32b-q4_K_M` (~22 GB @ 32k `num_ctx`) Fast Worker: `qwen3:8b` (~5.5 GB) Vision: `gemma4` (~8 GB) Embeddings: `bge-m3` (~1.2 GB)	4 parallel requests	~40–70 tok/s; concurrent orchestrator deliberation and 3 parallel research workers without model swapping.
Tier 3: CPU-Only / Low-VRAM Fallback	<12 GB VRAM or CPU-only VPS (32+ GB RAM)	Primary: `qwen3:8b-q4_K_M` (~5.5 GB)	1 sequential request	~6–12 tok/s; functional for low-volume scheduling, but multi-worker research loops will exhibit high latency (2–5 minutes per campaign).

4.4 Ephemeral Worker Sandboxing and Controlled Network Access

Requirement 7 of `idea.md` mandates dedicated, isolated execution environments for dynamically spawned workers with controlled tool access and **no unrestricted direct internet access**. Evaluating container isolation technologies reveals:

- **Standard Docker (`runc`)**: Shares the host Linux kernel directly; a container escape vulnerability in a headless browser (`Playwright`) or document parser exposes the host OS.
- **gVisor (`runsc`)**: An open-source OCI-compatible application kernel written in memory-safe Go by Google (`google/gvisor`, Apache 2.0 License). It intercepts guest application system calls in userspace before they reach the host Linux kernel and integrates directly with Docker (`docker run --runtime=runsc`) (gVisor Docker Quick Start). Note that per Mirantis gVisor documentation, `runsc` containers should be invoked with a TTY flag (`-t`) where applicable and do not require hardware virtualization extensions that may be unavailable on nested cloud VPS instances (unlike Firecracker/Kata).
- **Controlled Egress Architecture**: By attaching ephemeral `runsc` worker containers exclusively to an isolated Docker bridge network configured with `internal: true`, the Linux kernel blocks all direct outbound IP routing from the worker. The only reachable service on that bridge network is an internal **MCP Tool Gateway & Forward Proxy** (Envoy/Squid), which validates a short-lived HMAC capability token issued by the Master Orchestrator and restricts outbound requests strictly to allowlisted domains or authorized platform APIs.

4.5 Agentic RAG and Vector Storage (`PostgreSQL + pgvector`)

Introducing a separate standalone vector database (such as Milvus, Qdrant, or Weaviate) alongside PostgreSQL adds operational overhead, backup complexity, and dual-write consistency bugs. The pgvector extension (PostgreSQL License) provides native `vector` and `halfvec` data types with Hierarchical Navigable Small World (**HNSW**) graph indexing (`CREATE INDEX ON items USING hnsw (embedding vector_cosine_ops)`) directly inside PostgreSQL (Crunchy Data HNSW Guide). Combining `pgvector` HNSW semantic search with PostgreSQL native `tsvector` GIN full-text search enables single-query **Hybrid Search** with metadata filtering (`WHERE brand_id = $1 AND doc_type = $2`) and Reciprocal Rank Fusion (RRF) (Crunchy Data Hybrid Search), keeping relational state, LangGraph checkpoints, Temporal visibility tables, time-series analytics, and RAG embeddings unified in a single ACID database engine.

5. Feasibility Assessment: Capability Verification Matrix and Hardware-to-API Bottleneck Analysis

5.1 Comprehensive Capability Classification Matrix

Every major capability proposed in `idea.md` has been evaluated against official platform documentation and classified using the six mandatory verification states:

- **Verified**: Explicitly supported by official documentation, APIs, or open-source libraries.
- **Configurable**: Supported, but requires non-trivial infrastructure setup (e.g., public HTTPS media ingress, Docker `runsc` runtime configuration).
- **Custom Implementation**: Requires original software engineering to bridge components (e.g., `campaign.json` compiler, closed-loop anomaly evaluator).
- **Restricted**: Subject to external platform approvals, compliance audits, business verification, paid API tiers, or strict rate-limit ceilings.
- **Unverified**: Insufficient public evidence to guarantee deterministic behavior (e.g., autonomous paid ad bidding via small local LLMs).
- **Unavailable**: Explicitly unsupported by the external platform API (e.g., organic carousels via LinkedIn Posts API, video posts via Google Business Profile API).

Capability Area	Specific System Feature	Classification	Authoritative Evidence, Quotas, & Technical Constraints
Local Orchestration	Ollama tool calling & structured `campaign.json` compilation	**Verified**	Supported via Ollama Tool Calling and Structured Outputs; requires separating tool calls from JSON schema `format` calls (Issue #8095).
Worker Sandboxing	Isolated ephemeral workers without direct internet access	**Configurable**	Supported via Docker `--runtime=runsc` (gVisor) on an `internal: true` bridge network routed through an internal Tool Gateway proxy.
Media Generation	External AI image & video synthesis + local normalization	**Configurable**	Supported via Fal.ai and Replicate REST APIs (Flux, Kling, Luma) paired with local `FFmpeg`/`Pillow` transcoding.
LinkedIn Integration	Personal (`urn:li:person`) & Company Page	**Restricted**	Uses `POST https://api.linkedin.com/rest/posts` (LinkedIn Posts API). Personal requires `w_member_social`; Organization publishing (`w_organization_social`) & analytics (`r_organization_social`)

Capability Area	Specific System Feature	Classification	Authoritative Evidence, Quotas, & Technical Constraints
	(`urn:li:organization`) publishing		require **Community Management API** approval and `ADMINISTRATOR` / `CONTENT_ADMIN` role. **Unavailable**: Organic carousels (only `MultiImage` or `Documents` supported organically).
YouTube Integration	Video & Shorts publishing (`videos.insert`) and analytics	**Restricted**	Free API with 3 separate quota buckets since June 2026: **100 `videos.insert` calls/day**, 100 `search.list` calls/day, and 10,000 general units/day (YouTube Quota Guide 2026). **Critical Blocker**: Projects created after July 28, 2020 force all uploaded videos to `private` until the project passes the **YouTube API Services Compliance Audit**; unverified OAuth apps are capped at 100 users. Shorts require vertical/square aspect ratio `≤3 min` (no separate endpoint).
Instagram Integration	Feed images, Videos, Reels, Stories, and Carousels (`≤20` items)	**Restricted**	Uses `POST /<IG_ID>/media` and `POST /<IG_ID>/media_publish` (Meta Instagram Content Publishing). Requires Instagram Professional account, `instagram_business_basic` + `instagram_business_content_publish`, and Meta Business Verification / App Review for Advanced Access. **Rate Limit**: 50–100 API-published posts per 24 hours (queried via `/<IG_ID>/content_publishing_limit`). **Constraint**: Images must be **JPEG only** and hosted on a **public HTTPS server** so Meta can cURL them.
Threads Integration	Text (`≤500` chars), Image, Video (`≤5 min`), Carousels (`2–20` items), and Replies	**Restricted**	Dedicated API at `graph.threads.com` (Meta Threads API Overview) using `threads_basic`, `threads_content_publish`, `threads_manage_replies`. **Rate Limit**: 250 posts/24h, 1,000 replies/24h, 100 deletions/24h; total calls bounded by `4800 * max(10, Impressions)`. Media must be hosted at a public HTTPS URL.
Facebook Integration	Page Feed posts, Photos, Videos, Reels, and native scheduling	**Restricted**	Uses `POST /v26.0/{page_id}/feed`, `/photos`, and Video API (Facebook Pages API). Requires `pages_manage_posts`, `pages_read_engagement`, `pages_show_list`, `publish_video`, and Page `CREATE_CONTENT` task. Supports native scheduling (`published=false`, `scheduled_publish_time` 10 min to 30 days ahead). Note: Personal Facebook profile posting is **Unavailable** via API.
X (Twitter) Integration	Text (`≤280` chars), Threads, Polls, and Media (4 images, 1 video `≤512 MB`, or 1 GIF)	**Restricted**	Uses `POST https://api.x.com/2/tweets` and 3-step chunked upload (`initialize`, `append`, `finalize` on `/2/media/upload`) with OAuth 2.0 PKCE (`tweet.read`, `tweet.write`, `users.read`, `offline.access`) (X API Posting Guide 2026). **Commercial Constraint**: As of Feb 2026, new developer accounts use pay-per-use billing ($0.01/post, $0.005/read) or legacy Basic ($200/mo); rate limit is 100 posts/15 min per user.
Pinterest Integration	Pin creation (`POST /v5/pins`), Board creation, and Pin Analytics	**Restricted**	Uses Pinterest API v5 (Pinterest Access Tiers). **Critical Blocker**: All new apps start in **Trial Access**, where created Pins and Boards are **sandbox entities visible ONLY to the creator**. Public visibility requires submitting a video recording of the live OAuth flow and upgrading to **Standard Access**. Apps registered on/after Sept 14, 2026 receive HTTP 403 `PINNER_DATA_ACCESS_DENIED` on non-business accounts.
Google Business Profile	Local Posts (`STANDARD`, `EVENT`, `OFFER`, `ALERT`) and Review Replies	**Restricted**	Uses `POST https://mybusiness.googleapis.com/v4/accounts/{accountId}/locations/{locationId}/localPosts` with `business.manage` scope (GBP API 2026 Guide). **Critical Blocker**: Every project starts with **0 QPM quota** and **no sandbox**. Approval requires applying via the GBP Basic API Access form from an email managing a Business Profile **verified and active for 60+ days** with a listed website (unlocking 300 QPM). **Unavailable**: Video posts, Product posts, and Q&A API (discontinued Nov 3, 2025).
Closed-Loop Optimization	Autonomous content & schedule adjustment from live analytics	**Custom Implementation**	Telemetry ingestion is **Verified** via platform insights APIs; statistical evaluation and campaign diff generation require custom Python engineering with mandatory human approval to prevent feedback-loop instability.

5.2 Deep-Dive Technical Blockers and Architectural Mitigations

1. **The "Self-Hosted LAN vs. Public Media URL" Paradox**: - **Blocker**: Instagram Graph API (`image_url` / `video_url`), Threads API, and Google Business Profile API (`sourceUrl`) do not accept raw binary `multipart/form-data` payloads for standard image/post creation; instead, Meta and Google servers execute an outbound `cURL` request to fetch the media from a **publicly routable HTTPS URL**. A purely local MinIO container on `192.168.1.x` or `localhost` cannot be reached by Meta or Google servers. - **Mitigation**: The Media Library architecture implements a **Dual-Tier Storage & Ephemeral Staging Pipeline**. Master assets reside on local self-hosted MinIO storage. Immediately prior to publishing to pull-based APIs (Instagram, Threads, GBP, Pinterest URL source), the publishing activity either (a) exposes a single-file, random-UUID, short-TTL (15-minute) HTTPS path via an authenticated Cloudflare Tunnel / reverse proxy pointing only to the `/public-media-staging/` MinIO bucket, or (b) uploads the transcoded file to an encrypted S3/Cloudflare R2 staging bucket with an automated 1-hour lifecycle deletion rule.
2. **External Platform Compliance Audits and Approval Lead Times**: - **Blocker**: Even when all code is 100% complete, a newly created Google Cloud project cannot make a single GBP API call (`0 QPM`) until approved (requiring a 60-day verified business profile), cannot publish public YouTube videos (`private` lock until Compliance Audit completion), and cannot publish public Pinterest Pins (Trial Access sandbox visibility until video demo approval). - **Mitigation**: Decouple core platform engineering from external review queues by implementing a **Contract-Compliant Mock Platform Server** for local integration testing and designing each platform adapter with an explicit `AccountEligibilityState` (`UNCONFIGURED`, `SANDBOX_RESTRICTED`, `PRODUCTION_VERIFIED`). When an account is in `SANDBOX_RESTRICTED` mode (e.g., Pinterest Trial or YouTube un-audited), the UI and Orchestrator explicitly warn the operator that published assets will land as private/sandbox items and automatically record the required audit demo artifacts.

6. Integration Specifications: OAuth 2.0 Flows, End-to-End API Contracts, and Verification Gates Across Eight Platforms

6.1 Unified Integration Pipeline and Security Architecture

Every external platform integration executes as an isolated, deterministic **Temporal Activity** adhering to a strict seven-stage contract:

Contract Pipeline: `Input Blueprint` → `Vault Token Decryption` → `Pre-Flight Scope & Quota Check` → `Media Staging & API Execution` → `Schema & Status Validation` → `PostgreSQL Persistence` → `Read-After-Write Verification`

- **Credential Vault & Token Refresh**: OAuth 2.0 credentials are stored in the `social_credentials` table encrypted via `AES-256-GCM` using a master key injected via Docker Secret / KMS (`VAULT_MASTER_KEY`) and per-row unique initialization vectors (`nonce`). A dedicated Temporal cron workflow (`token_refresh_workflow`) runs every 15 minutes to proactively rotate tokens before expiration (e.g., X OAuth 2.0 tokens expire in 2 hours; Google access tokens expire in 1 hour; Meta and LinkedIn long-lived tokens expire in 60 days).
- **Distinguishing Connection vs. Verified Publishing**: A green "Connected" badge in the UI only indicates that an OAuth token was issued (**Authentication Success**) and holds the expected scope strings (**Authorization Success**). A publishing task is only marked `VERIFIED_PUBLISHED` when (1) the platform API returns HTTP `200/201` with a resource identifier (**API Execution Success**) AND (2) a subsequent polling or read-after-write `GET` request confirms the resource exists on the external platform in a `PUBLISHED` or `LIVE` state with a valid permalink (**Verified External Side Effect**).

6.2 Platform-by-Platform API Contracts and Verification Specifications

Platform	API Version & Base Host	Auth Mechanism & Required Scopes	Token Li Policy
LinkedIn	Posts API `https://api.linkedin.com/rest` Headers: `Linkedin-Version: 202610`, `X-Restli-Protocol-Version: 2.0.0`	OAuth 2.0 (3-legged) Scopes: `w_member_social`, `w_organization_social`, `r_organization_social`, `r_member_social` (restricted)	Access t Refresh approve user 7 d re-auth r
YouTube	YouTube Data API v3 `https://www.googleapis.com/upload/youtube/v3`	OAuth 2.0 Scopes: `youtube.upload`, `youtube.readonly`, `yt-analytics.readonly`	Access t hour). Refresh (access_ prompt=
Instagram	Instagram Platform API `https://graph.facebook.com/v26.0` & `rupload.facebook.com`	OAuth 2.0 (Facebook Login for Business / Instagram Login) Scopes: `instagram_basic`, `instagram_content_publish`, `pages_read_engagement`	Short-liv for 60-d or Syste refresh a
Threads	Threads API v1.0 `https://graph.threads.com/v1.0`	OAuth 2.0 Scopes: `threads_basic`, `threads_content_publish`, `threads_manage_replies`, `threads_manage_insights`	60-day l refresh v /refres before d
Facebook Pages	Pages API v26.0 `https://graph.facebook.com/v26.0`	OAuth 2.0 Scopes: `pages_manage_posts`, `pages_read_engagement`, `pages_read_user_engagement`, `pages_show_list`, `publish_video`	Long-live exchang for non- Token.
X (Twitter)	X API v2 `https://api.x.com/2`	OAuth 2.0 with PKCE Scopes: `tweet.read`, `tweet.write`, `users.read`, `offline.access`	Access t **(7,200s)** Refresh `POST /2` to every `min` rema
Pinterest	Pinterest API v5 `https://api.pinterest.com/v5`	OAuth 2.0 Scopes: `boards:read`, `boards:write`, `pins:read`, `pins:write`, `user_accounts:read`	Access t Refresh (continu
Google Business Profile	GBP APIs v1 / v4 `https://mybusiness.googleapis.com/v4`	OAuth 2.0 Scope: `https://www.googleapis.com/auth/business.manage`	Access t hour). Refresh (access_

6.3 Unified Error-Handling and Resilience Strategy

All platform adapters map HTTP and platform-specific error payloads into four canonical Temporal retry categories:

1. **`TransientNetworkOrServerError` (HTTP `500`, `502`, `503`, `504`, LinkedIn `409 CONFLICT`)**: Retried automatically by Temporal with exponential backoff (initial interval `5s`, coefficient `2.0`, max attempts `5`), preserving the upload session URI (for YouTube HTTP `308`) or idempotency key.
2. **`RateLimitOrQuotaExhaustedError` (HTTP `429`, YouTube HTTP `403 quotaExceeded`, Meta subcode `2207042` / error `368`, GBP `429 RESOURCE_EXHAUSTED`)**: **Non-immediate retry**. Immediately halts retries for that platform/account bucket, parses `x-rate-limit-reset` or schedules a Temporal timer until the next quota window (e.g., 24 hours for YouTube `quotaExceeded` or LinkedIn 429 cooldown), and emits a high-priority workspace alert.
3. **`AuthenticationOrPermissionError` (HTTP `401`, Meta error `190`, LinkedIn `403 ACCESS_DENIED`, Pinterest `403 PINNER_DATA_ACCESS_DENIED`)**: Triggers a single immediate token refresh attempt via `token_refresh_activity`. If refresh fails or scope is missing, marks the `social_credentials` row as `REAUTH_REQUIRED`, pauses queued jobs for that channel, and notifies the operator.
4. **`SemanticOrPolicyValidationError` (HTTP `400`, `422 UNPROCESSABLE_ENTITY`, GBP `state == REJECTED`)**: Non-retriable by the network adapter. Routes the structured error reason back to a LangGraph repair node so the Master Orchestrator can propose a compliant revision (e.g., stripping an unverified phone number from a GBP post summary or shortening an X post) for human sign-off.

7. Recommended Technology Stack: Justified Production Selections and Rejected Alternatives

In accordance with the operating principle of **Simplicity Before Overengineering**, every component in the technology stack is selected to satisfy validated functional and security requirements while minimizing moving parts on a single self-hosted server.

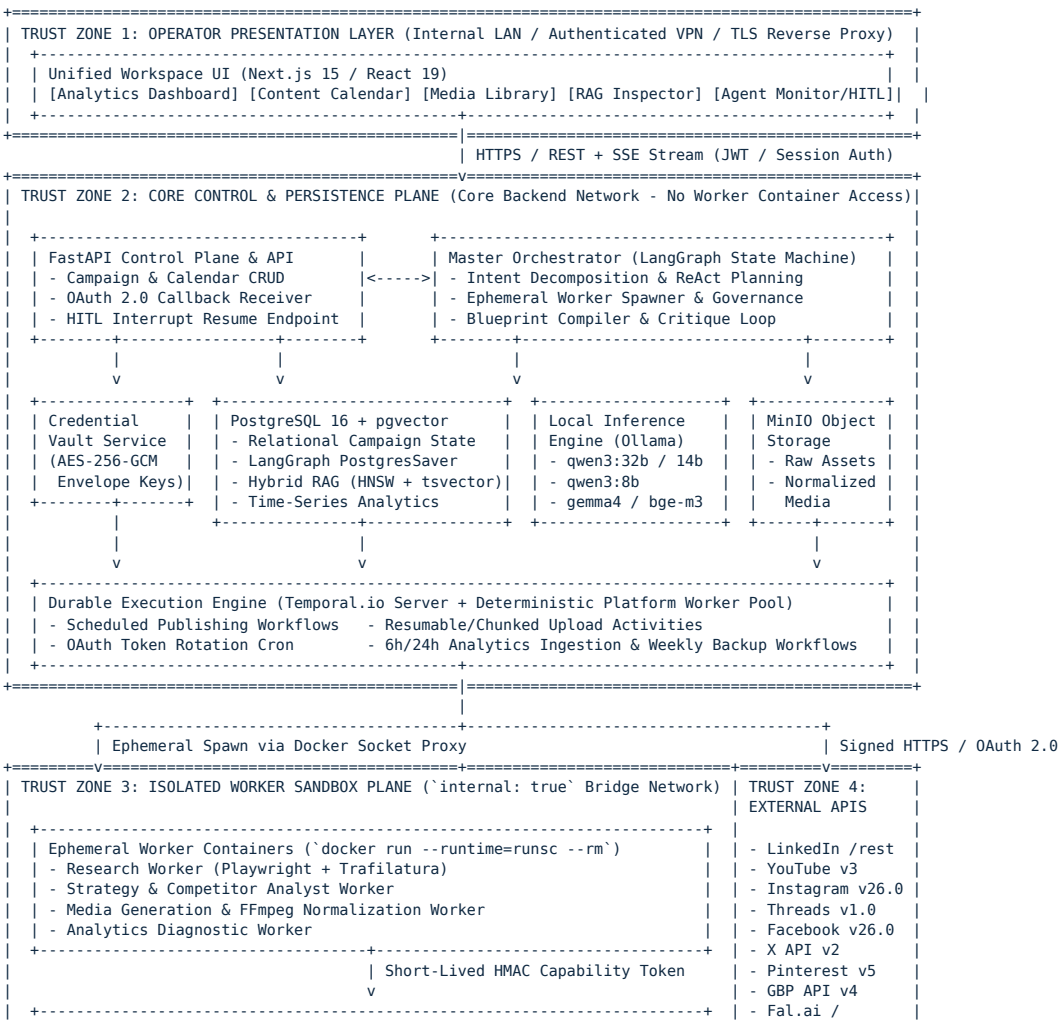
Architectural Layer	Selected Technology	License	Technical Justification	Rejected Alternatives & Reason for Rejection
Cognitive Agent Orchestrator	**Python 3.12 + LangGraph** (`langgraph>=1.2.12`, `langgraph-checkpoint-postgres`)	MIT	Explicit state schemas, cyclical ReAct graphs, native `PostgresSaver` checkpointing, and typed `interrupt()` / `Command(resume=...)` human-in-the-loop primitives (LangGraph Interrupts).	**AutoGen / CrewAI**: Higher prompt overhead, less deterministic state checkpointing, and weaker native database-backed pause/resume semantics for multi-day human approvals.
Local LLM & Embeddings	Ollama (`qwen3:32b/14b`, `qwen3:8b`, `gemma4`, `bge-m3`) + **Pydantic v2**	MIT	Native GGUF quantization management, REST `/api/chat` tool calling, `think=True` traces, and grammar-constrained JSON schema output (`format`).	**vLLM / TGI**: Higher baseline VRAM footprint when multiplexing multiple model roles (chat, vision, embeddings) on a single workstation GPU.

Architectural Layer	Selected Technology	License	Technical Justification	Rejected Alternatives & Reason for Rejection
Durable Workflow & Scheduling Engine	**Temporal.io** (Server + Python SDK `temporalio`)	MIT	Proven in Postiz; provides crash-proof multi-week scheduling timers, chunked media upload heartbeating, and per-platform rate-limit queues using the existing PostgreSQL instance.	**Celery / BullMQ / APScheduler**: Lack durable multi-step execution state across long delays and require brittle custom state tables to recover interrupted multi-step container uploads (Celery vs Temporal).
Backend API & Control Plane	**FastAPI** (Python 3.12, `uvicorn`, `httpx`, `SQLAlchemy 2.0` async)	MIT	Shares Pydantic v2 schemas directly with LangGraph, Ollama Structured Outputs, and Temporal Activities, eliminating cross-language schema drift.	**NestJS (TypeScript)**: Would require maintaining duplicate Zod and Pydantic schemas across the Python AI/RAG layer and TypeScript API layer.
Unified Database & Agentic RAG	**PostgreSQL 16 + pgvector** (`0.8+`)	PostgreSQL	Unifies relational state, LangGraph checkpoints, Temporal persistence, time-series post metrics, and hybrid `tsvector` + HNSW vector search (Crunchy Data Hybrid Search) in one engine.	**Qdrant / Milvus / Weaviate + MongoDB**: Introduces 2–3 extra stateful database containers, increasing RAM consumption and backup synchronization complexity.
Worker Sandbox & Egress Control	**Docker Engine + gVisor (`runsc`) + Envoy Forward Proxy**	Apache 2.0	Userspace application kernel isolates untrusted browser/crawler execution from the host Linux kernel; `internal: true` bridge network + Envoy proxy enforces strict tool/domain allowlists.	**Unisolated Subprocesses (`subprocess.Popen`)**: Violates Requirement 7 isolation; **Firecracker microVMs**: Requires bare-metal `/dev/kvm` access often unavailable on standard cloud VPS instances.
Web Research & Crawling Tools	**Playwright (Headless Chromium) + Trafilatura + SearXNG (Self-Hosted)**	Apache 2.0 / AGPL-3.0	Self-hosted SearXNG provides zero-cost meta-search JSON APIs; Playwright + Trafilatura inside `runsc` workers extract clean Markdown from JS-heavy competitor pages.	**Paid SERP SaaS Only**: Violates local-first autonomy when a self-hosted metasearch engine can satisfy baseline discovery.
Media Processing & Storage	**MinIO (S3-compatible) + FFmpeg + Pillow + Fal.ai / Replicate SDKs**	AGPL-3.0 / LGPL / MIT	Local S3-compatible object storage with deterministic FFmpeg video normalization (H.264/AAC, aspect ratio padding, `moov` faststart) and pluggable external generation adapters.	**Local Filesystem Directories Only**: Lacks presigned URL generation required for pull-based social APIs (Instagram, Threads, GBP).
Unified Workspace Frontend	**Next.js 15 (React 19, TypeScript, Tailwind CSS, TanStack Query, FullCalendar)**	MIT	Industry-standard reactive UI with Server-Sent Events (SSE) for live LangGraph telemetry, interactive calendar drag-and-drop, and schema-driven HITL approval forms.	**Streamlit / Gradio**: Insufficient layout flexibility for a unified multi-calendar, media asset library, and real-time agent monitoring workspace.

8. System Architecture: Control Plane, Sandboxed Worker Data Plane, and Closed-Loop Analytics

8.1 High-Level Component Topology and Trust Boundaries

The system architecture is partitioned into four explicit **Trust Zones** enforced by Docker network policies and cryptographic capability tokens:



```
|  | Internal MCP Tool Gateway & Envoy Egress Proxy                          |  |   Replicate APIs  |
|  | - Enforces Per-Worker Tool Permissions & Domain Allowlists              +->| - Public Signed   |
|  | - Routes Search Queries to Self-Hosted SearXNG                          |  |   Media Staging   |
|  +-------------------------------------------------------------------------+  +-------------------+
+=============================================================================================================+
```

8.2 Master Orchestrator State Machine and Dynamic Ephemeral Worker Lifecycle

In accordance with Requirement 1 and Section 6 principles, agents are deployed **strictly where dynamic deliberation and unstructured synthesis add value** (market research, competitor positioning, creative copy/prompt formulation, and performance anomaly diagnosis), while predictable operations (media transcoding, OAuth token refresh, scheduled publishing, and metrics polling) run as deterministic Temporal Activities.

The Master Orchestrator is implemented as a compiled `StateGraph` in LangGraph with the following nodes and edges:

1. **`ingest_directive_node`**: Parses user intent, queries the pgvector hybrid store for brand guidelines and historical performance baselines, and initializes the typed `OrchestratorState`.
2. **`plan_research_tasks_node`**: Uses `qwen3:32b` (or `14b`) with `think=True` to decompose the directive into a directed acyclic graph (DAG) of research tasks.
3. **`spawn_ephemeral_workers_node` (Parallel Fan-Out via `Send` API)**: - For each research task, the orchestrator requests an ephemeral container from the **Sandbox Manager** via a restricted Docker Socket Proxy (`tecnativa/docker-socket-proxy` permitting only `CONTAINERS` and `NETWORKS`, blocking `PRIVILEGED` host mounts). - The container boots from a read-only root filesystem (`--read-only --tmpfs /tmp:rw,noexec,nosuid,size=256m --cap-drop=ALL --security-opt=no-new-privileges --runtime=runsc --memory=1g --cpus=1.0`) attached solely to `sandbox_internal_net`. - The orchestrator injects a single-use HMAC capability token (`WORKER_CAPABILITY_TOKEN`) scoping which tools on the **Internal MCP Tool Gateway** the worker may invoke (e.g., `searxng_search`, `playwright_extract_markdown`, `rag_read_brand_docs`) and setting a hard budget (`max_tool_calls=8`, `ttl_seconds=180`). - Once the worker returns its structured JSON finding, the `finally` block executes `container.remove(force=True)` and revokes the HMAC token.
4. **`synthesize_strategy_and_interrupt_node` (`HITL-01`)**: Consolidates worker findings into a cited Strategy Brief and calls `interrupt(StrategyApprovalPayload, response_schema=StrategyDecision)`.
5. **`compile_campaign_blueprint_node`**: Runs a dedicated Ollama Structured Outputs call (`format=CampaignBlueprint.model_json_schema()`, `temperature=0`) to generate `campaign.json`, followed by a deterministic Python `PlatformConstraintValidator` node. If any platform constraint fails (e.g., an X post is 310 characters or a LinkedIn post requests an unsupported organic carousel), a conditional edge loops back to `repair_blueprint_node` (max 3 iterations).
6. **`orchestrate_media_workers_node`**: Spawns Media Workers to call Fal.ai / Replicate via the Tool Gateway, download assets to MinIO, and execute `FFmpeg` / `Pillow` normalization and `gemma4` vision quality checks (verifying no garbled text or aspect-ratio clipping).
7. **`await_campaign_publish_approval_node` (`HITL-02`)**: Calls `interrupt(CampaignPreviewPayload, response_schema=CampaignApprovalDecision)`.
8. **`dispatch_to_temporal_node`**: Upon human approval, commits the immutable `campaign.json` snapshot to PostgreSQL and starts the deterministic `CampaignExecutionWorkflow` in Temporal.io.

8.3 End-to-End Sequence Flow: Campaign Creation to Verified Publishing

```
Operator          Workspace UI       FastAPI / LangGraph       gVisor Worker + Proxy       Temporal + Adapters       Social APIs
   |                   |                    |                          |                          |                    |
   |--1. Goal Prompt-->|                    |                          |                          |                    |
   |                   |--2. POST /campaigns->|                        |                          |                    |
   |                   |                    |--3. Hybrid RAG Lookup--->|                          |                    |
   |                   |                    |--4. Spawn `runsc`------->|                          |                    |
   |                   |                    |     Research Workers     |--5. Allowlisted Crawl--->|                    |
   |                   |                    |<--6. JSON Findings-------|     (SearXNG/Playwright) |                    |
   |                   |                    |     (Destroy Container)  |                          |                    |
   |                   |                    |--7. Compile campaign.json|                          |                    |
   |                   |                    |     & Generate Media---->|--8. Fal.ai + FFmpeg----->|                    |
   |                   |<--9. SSE `interrupt`-|                        |                          |                    |
   |<--10. Render HITL-|      (HITL-02)     |                          |                          |                    |
   |--11. Approve----->|--12. Command(resume)>|                        |                          |                    |
   |                   |                    |--13. Start Durable Workflow----------------------->|                    |
   |                   |                    |                          |                          |--14. Sleep until--->|
   |                   |                    |                          |                          |      scheduled_at   |
   |                   |                    |                          |                          |--15. Refresh Token->|
   |                   |                    |                          |                          |--16. Upload & Post->|
   |                   |                    |                          |                          |<--17. HTTP 201 + ID-|
   |                   |                    |                          |                          |--18. Read-After---->|
   |                   |                    |                          |                          |      Write GET      |
   |                   |<--19. Update Status `VERIFIED_PUBLISHED` + Live Permalink-----------------|                    |
```

8.4 Closed-Loop Analytics and Autonomous Optimization Engine

To satisfy Requirement 6 without allowing stochastic LLM behavior to corrupt live campaigns, the **Observe–Analyze–Act–Evaluate** loop combines deterministic statistical evaluation with bounded agentic investigation:

1. **Observe (Deterministic Temporal Cron)**: Every 6 hours for active campaigns (and every 24 hours for historical posts up to 30 days old), `AnalyticsIngestionWorkflow` queries platform insights endpoints (`GET /rest/organizationalEntityShareStatistics` on LinkedIn, `GET /<IG_MEDIA_ID>/insights` on Instagram, `GET /{threads-media-id}/insights` on Threads, YouTube Analytics API, Pinterest `GET /v5/pins/{pin_id}/analytics`, X `GET /2/tweets` with `public_metrics`). Every Sunday at `02:00 UTC`, `WeeklyBackupWorkflow` exports an immutable compressed Parquet + JSONL snapshot of all analytics, execution logs, and task states to MinIO (`/backups/weekly/YYYY-WW/`).
2. **Analyze (Deterministic Z-Score & Cohort Evaluator)**: After ingestion, a Python statistical service computes normalized engagement efficiency metrics per platform and content pillar:

Engagement Velocity Formula: $EV = [\ (Likes + 2 \times Comments + 3 \times Shares/Saves) / \max(Impressions, 100)\ ] \times (24 / \Delta t_{hours})$

It compares each post's EV against the rolling 30-day platform baseline ($\mu_{platform}$, $\sigma_{platform}$), flagging **Underperforming Outliers** ($Z < -1.5$) and **High-Performing Outliers** ($Z > +1.5$). 3. **Act (Agentic Diagnostic Worker + Blueprint Adjustment)**: When a campaign cohort exhibits statistically significant divergence (minimum sample size $N \geq 5$ published posts), Temporal triggers the `OptimizationSubgraph` in LangGraph. The Master Orchestrator spawns an `AnalyticsDiagnosticWorker` to correlate outlier posts against posting time-of-day, media format (e.g., Reel vs. Carousel vs. static image), hook length, and competitor activity. The orchestrator drafts a structured `OptimizationProposal` containing concrete adjustments for the next campaign cycle's `campaign.json` (e.g., shifting LinkedIn publishing windows

or reallocating creative ratios) and pauses at `HITL-03` for operator approval. 4. **Evaluate (Knowledge Base Retention)**: Seven days after an approved optimization is applied, the system computes the post-intervention cohort delta (ΔEV) and writes a cited `CampaignRetrospective` document into the `pgvector` knowledge base so future `ingest_directive_node` runs automatically retrieve proven platform-specific learnings.

9. Implementation Specification: Monorepo Topology, PostgreSQL/pgvector Schemas, and JSON Contracts

9.1 Repository and Module Organization

The project is organized as a clean, reproducible monorepo managed via `uv` (Python workspace) and `pnpm` (Next.js workspace), orchestrated via `docker-compose.yml`:

```
agentic-marketing-os/
├── docker-compose.yml                  # Postgres+pgvector, Temporal, MinIO, SearXNG, Envoy, API, Web
├── .env.example                        # Documented environment variables & secrets template
├── infra/
│   ├── envoy/envoy.yaml                # Worker egress forward proxy & domain allowlist config
│   ├── gvisor/daemon.json              # Docker runsc OCI runtime registration
│   └── postgres/init-pgvector.sql      # pgvector, pg_trgm, and uuid-ossp initialization
├── services/
│   ├── control-plane/                  # FastAPI + LangGraph + Temporal Workers (Python 3.12)
│   │   ├── pyproject.toml
│   │   ├── app/
│   │   │   ├── main.py                 # FastAPI entrypoint, REST + SSE streaming endpoints
│   │   │   ├── config.py               # Pydantic Settings environment validation
│   │   │   ├── security/
│   │   │   │   ├── vault.py            # AES-256-GCM envelope encryption for OAuth tokens
│   │   │   │   └── capability.py       # Short-lived HMAC token signer for ephemeral workers
│   │   │   ├── db/
│   │   │   │   ├── models.py           # SQLAlchemy 2.0 ORM models (campaigns, posts, rag, metrics)
│   │   │   │   └── migrations/         # Alembic schema migrations
│   │   │   ├── rag/
│   │   │   │   └── hybrid_store.py     # pgvector HNSW + tsvector RRF retrieval service
│   │   │   ├── orchestrator/
│   │   │   │   ├── state.py            # LangGraph TypedDict & Pydantic state definitions
│   │   │   │   ├── graph.py            # Master Orchestrator StateGraph & HITL interrupt nodes
│   │   │   │   ├── ollama_client.py    # Strict separation of tool-calling vs schema-format calls
│   │   │   │   └── sandbox_manager.py  # Docker SDK `runsc` ephemeral container lifecycle manager
│   │   │   ├── mcp_gateway/
│   │   │   │   └── server.py           # Internal tool execution gateway for sandboxed workers
│   │   │   ├── media/
│   │   │   │   ├── generators.py       # Fal.ai & Replicate async polling clients
│   │   │   │   └── normalizer.py       # FFmpeg & Pillow deterministic transcoding & validation
│   │   │   └── workflows/
│   │   │       ├── schedules.py        # Temporal workflows (Publishing, TokenRefresh, Analytics)
│   │   │       └── adapters/           # 8 Direct Platform Adapters
│   │   │           ├── base.py         # Abstract PlatformAdapter & 7-stage contract interface
│   │   │           ├── linkedin.py     # LinkedIn /rest/posts & Images/Videos/Documents API
│   │   │           ├── youtube.py      # YouTube Data API v3 resumable upload & audit guard
│   │   │           ├── instagram.py    # Instagram Graph API v26.0 container & media_publish
│   │   │           ├── threads.py      # Threads API v1.0 container & threads_publish
│   │   │           ├── facebook.py     # Facebook Pages API v26.0 feed, photos, & video_reels
│   │   │           ├── x_twitter.py    # X API v2 chunked media upload & /2/tweets
│   │   │           ├── pinterest.py    # Pinterest API v5 /v5/pins & Trial sandbox detector
│   │   │           └── gbp.py          # Google Business Profile v4 localPosts & policy sanitizer
│   │   └── tests/                      # Unit, contract, sandbox security, and E2E test suites
│   └── worker-sandbox-image/           # Hermetic Dockerfile built for `--runtime=runsc`
│       ├── Dockerfile
│       └── worker_entrypoint.py        # Scoped worker loop communicating only with mcp_gateway
└── apps/
    └── workspace-web/                  # Next.js 15 Unified Workspace UI (TypeScript)
        ├── package.json
        └── src/
            ├── app/                    # Dashboard, Calendar, Media, Knowledge, Campaigns, Monitor
            └── components/             # HITL Approval Modal, SSE Trace Viewer, Calendar Grid
```

9.2 Core Database Schema (`PostgreSQL 16 + pgvector`)

The database schema unifies relational campaign tracking, cryptographic token storage, time-series telemetry, and hybrid RAG vectors.

Table Name	Primary Key & Indexes	Key Columns & Types	Purpose & Integrity Constraints
`brands`	`id UUID PK`	`name TEXT`, `domain TEXT`, `voice_guidelines JSONB`, `created_at TIMESTAMPTZ`	Top-level tenant/brand workspace boundary.
`social_credentials`	`id UUID PK` `UNIQUE(brand_id, platform, external_account_id)`	`brand_id UUID FK`, `platform VARCHAR(32)`, `external_account_id TEXT`, `account_urn TEXT`, `encrypted_access_token BYTEA`, `encrypted_refresh_token BYTEA`, `encryption_nonce BYTEA`, `scopes TEXT[]`, `eligibility_state VARCHAR(32)`, `expires_at TIMESTAMPTZ`	Stores `AES-256-GCM` encrypted OAuth tokens and tracks `UNCONFIGURED`, `SANDBOX_RESTRICTED`, or `PRODUCTION_VERIFIED` status.
`campaigns`	`id UUID PK` `INDEX(brand_id, status)`	`brand_id UUID FK`, `title TEXT`, `status VARCHAR(32)`, `blueprint_json JSONB`, `blueprint_version INT`, `langgraph_thread_id TEXT`, `approved_by TEXT`, `approved_at TIMESTAMPTZ`	Stores immutable `campaign.json` revisions and links to the LangGraph checkpoint `thread_id` and human approval audit record.
`media_assets`	`id UUID PK` `UNIQUE(sha256_hash)`	`brand_id UUID FK`, `campaign_id UUID FK`, `asset_type VARCHAR(16)`, `minio_object_key TEXT`, `public_staging_url TEXT`, `staging_expires_at TIMESTAMPTZ`, `width INT`, `height INT`, `duration_sec FLOAT`, `codec TEXT`, `file_size_bytes BIGINT`, `generation_provenance JSONB`	Central Media Library catalog with verified `ffprobe`/Pillow dimensions and ephemeral staging URL lifecycle tracking.
`scheduled_posts`	`id UUID PK` `UNIQUE(idempotency_key)`	`campaign_id UUID FK`, `credential_id UUID FK`, `platform VARCHAR(32)`, `idempotency_key CHAR(64)`, `scheduled_at`	Tracks every individual platform post from `DRAFT` → `APPROVED` → `QUEUED` →

Table Name	Primary Key & Indexes	Key Columns & Types	Purpose & Integrity Constraints
	INDEX(scheduled_at, status)	TIMESTAMPTZ, payload_json JSONB, media_asset_ids UUID[], status VARCHAR(32), temporal_workflow_id TEXT, external_post_id TEXT, external_permalink TEXT, verified_at TIMESTAMPTZ, last_error JSONB	PUBLISHED_UNVERIFIED → VERIFIED_PUBLISHED (or FAILED / SANDBOX_ONLY).
post_metrics_timeseries	PRIMARY KEY (post_id, captured_at)	post_id UUID FK, captured_at TIMESTAMPTZ, impressions BIGINT, reach BIGINT, likes INT, comments INT, shares INT, saves INT, clicks INT, engagement_velocity FLOAT, raw_platform_payload JSONB	Append-only time-series telemetry for closed-loop Z-score evaluation and weekly Parquet backups.
knowledge_chunks	id UUID PK HNSW(embedding vector_cosine_ops) GIN(fts_vector)	brand_id UUID FK, doc_type VARCHAR(32), source_uri TEXT, content TEXT, metadata JSONB, embedding VECTOR(1024), fts_vector TSVECTOR, created_at TIMESTAMPTZ	Hybrid RAG store indexing brand guidelines, competitor reports, and campaign retrospectives (bge-m3 1024-dim embeddings).
worker_execution_audit	id UUID PK INDEX(langgraph_thread_id)	worker_id TEXT, container_id TEXT, worker_role VARCHAR(32), spawned_at TIMESTAMPTZ, terminated_at TIMESTAMPTZ, tool_calls_json JSONB, exit_status VARCHAR(32)	Immutable audit trail of every dynamically spawned runsc container, its tool invocations, and verified teardown timestamp.

9.3 Canonical `campaign.json` Contract Specification

The Master Orchestrator compiles and validates `campaign.json` against the following strict schema before any media generation or scheduling occurs:

```
{
  "$schema": "https://agentic-marketing.local/schemas/campaign-v1.json",
  "campaign_id": "cmp_2026_11_q4_launch",
  "schema_version": "1.0.0",
  "brand_id": "8f9d2a10-4c3b-4e1a-9f8a-1b2c3d4e5f60",
  "strategy_reference_id": "know_7c6b5a40-3f2e-1d0c-8b7a-6e5f4d3c2b1a",
  "objective": {
    "primary_kpi": "engagement_velocity",
    "target_delta_pct": 15.0,
    "target_audience_segments": ["B2B SaaS Founders", "Self-Hosted Infrastructure Engineers"],
    "core_value_proposition": "Zero-lock-in marketing automation with verifiable data sovereignty."
  },
  "schedule_window": {
    "start_date": "2026-11-05T00:00:00Z",
    "end_date": "2026-11-19T23:59:59Z",
    "timezone": "UTC"
  },
  "media_requirements": [
    {
      "asset_ref": "asset_hero_diagram_01",
      "media_type": "IMAGE",
      "provider": "fal_flux_pro",
      "prompt": "Minimalist isometric architecture diagram of an autonomous server rack, high contrast, clean composition, text-free, no
watermarks",
      "target_specs": {
        "format": "JPEG",
        "width": 1080,
        "height": 1350,
        "aspect_ratio": "4:5",
        "max_bytes": 5242880
      }
    }
  ],
  "platform_posts": [
    {
      "slot_id": "slot_li_01",
      "platform": "LINKEDIN",
      "author_urn": "urn:li:organization:5515715",
      "scheduled_at": "2026-11-05T14:00:00Z",
      "post_type": "SINGLE_IMAGE",
      "copy": {
        "commentary": "Why decoupling cognitive planning from durable execution matters in self-hosted AI systems. #SoftwareArchitecture
#SelfHosted",
        "title": "Hybrid Agentic Architecture Blueprint",
        "cta_label": "LEARN_MORE",
        "landing_url": "https://example.org/architecture"
      },
      "media_refs": ["asset_hero_diagram_01"],
      "validation_constraints": {
        "max_chars": 3000,
        "requires_public_url": false
      }
    }
  ]
}
```

9.4 Internal REST and SSE API Endpoints

HTTP Method & Endpoint	Request Payload / Params	Response Contract	Architectural Responsibility
POST /api/v1/campaigns/orchestrate	{ "brand_id": UUID, "directive": str }	{ "thread_id": str, "campaign_id": UUID, "status": "PLANNING" }	Initializes a new LangGraph thread_id and starts the Master Orchestrator asynchronously.
GET /api/v1/orchestrator/{thread_id}/stream	Query: last_event_id	text/event-stream (SSE: node_transition, worker_spawned, token_delta, interrupt_raised)	Streams live LangGraph v3 events (stream.messages, stream.values, stream.interrupts) to the Agent Monitoring UI.

HTTP Method & Endpoint	Request Payload / Params	Response Contract	Architectural Responsibility
`POST /api/v1/orchestrator/{thread_id}/resume`	`{ "interrupt_id": str, "decision": "approve" | "edit" | "reject", "payload": dict }`	`{ "thread_id": str, "status": "RESUMED" | "DISPATCHED" }`	Validates human input against `Interrupt.response_schema`, writes an immutable row to `approval_audit_log`, and invokes `Command(resume=...)`.
`GET /api/v1/oauth/{platform}/authorize`	Query: `brand_id`	`{ "authorization_url": str, "state": str }`	Generates PKCE `code_challenge` and cryptographic `state` token for the target social platform OAuth 2.0 flow.
`GET /api/v1/oauth/{platform}/callback`	Query: `code`, `state`	HTTP 302 Redirect to Workspace Settings	Exchanges code for tokens, inspects granted scopes and platform eligibility (e.g., GBP quota `>0`, Pinterest tier), and encrypts tokens in `social_credentials`.
`GET /api/v1/analytics/overview`	Query: `brand_id`, `window_days`	`{ "kpis": {...}, "outliers": [...], "proposals": [...] }`	Serves aggregated cross-platform telemetry, cohort Z-scores, and pending `OptimizationProposal` diffs.

9.5 Environment Variables, Secrets, and Runtime Configuration Contract

Every service in the self-hosted stack validates its configuration at startup using Pydantic `BaseSettings`. Missing or malformed cryptographic keys immediately abort container initialization rather than falling back to insecure defaults.

Variable Name	Service Scope	Required Format / Default	Architectural & Security Purpose
`VAULT_MASTER_KEY`	`control-plane`, `temporal-worker`	64-character hex string (256-bit key) injected via Docker Secret	Root key-encryption-key (KEK) for `AES-256-GCM` envelope encryption of `social_credentials` rows.
`WORKER_HMAC_SECRET`	`control-plane`, `mcp-gateway`	64-character hex string rotated on deployment	Signs short-lived capability tokens (`TTL=180s`) authenticating `runsc` workers to the internal MCP Tool Gateway.
`OLLAMA_BASE_URL`	`control-plane`	`http://ollama:11434`	Internal Docker DNS endpoint for the local Ollama inference server.
`OLLAMA_ORCHESTRATOR_MODEL`	`control-plane`	`qwen3:32b-q4_K_M` (or `qwen3:14b-q4_K_M` on 24 GB VRAM)	Primary model for LangGraph planning, tool selection, and `campaign.json` structured compilation.
`OLLAMA_WORKER_MODEL`	`control-plane`, `mcp-gateway`	`qwen3:8b-q4_K_M`	High-throughput extraction model for ephemeral competitor analysis and summarization workers.
`OLLAMA_VISION_MODEL`	`control-plane`	`gemma4:latest`	Multimodal inspection model verifying generated images and video keyframes against campaign layout rules.
`OLLAMA_EMBED_MODEL`	`control-plane`	`bge-m3:latest` (1024 dimensions)	Dense embedding generator for `knowledge_chunks` HNSW indexing in `pgvector`.
`OLLAMA_NUM_PARALLEL`	`ollama`	`2` (24 GB VRAM) or `4` (48 GB VRAM)	Bounds concurrent KV-cache allocations to prevent GPU out-of-memory errors during worker fan-out.
`DATABASE_URL`	`control-plane`, `temporal-server`	`postgresql+asyncpg://user:pass@postgres:5432/marketing_os`	Primary ACID store for relational tables, `PostgresSaver` checkpoints, and `pgvector` hybrid search.
`PUBLIC_MEDIA_STAGING_BASE_URL`	`temporal-worker`	`https://media-stage.yourdomain.tld`	Publicly routable TLS ingress serving 15-minute signed URLs for Meta (`/<IG_ID>/media`), Threads, and GBP `localPosts`.

10. Implementation Roadmap: Dependency-Ordered Vertical Slices and Engineering Backlog

The implementation roadmap is structured into **six sequential, independently verifiable milestones**, starting with the smallest end-to-end vertical slice (**Milestone 1**) that proves local Ollama orchestration, human-in-the-loop checkpointing, Temporal scheduling, and verified publishing to a real external platform.

Milestone	Scope & Engineering Deliverables	Technical Dependencies	Primary Risk Mitigated
M1: Core Vertical Slice (Local Orchestrator → HITL → LinkedIn & X Text/Image Publish)	1. Docker Compose stack (`PostgreSQL 16 + pgvector`, `Temporal`, `MinIO`, `Ollama`, `FastAPI`). 2. `AES-256-GCM` Credential Vault + OAuth 2.0 PKCE/3-legged flows for **LinkedIn** and **X**. 3. LangGraph Master Orchestrator compiling `campaign.json` via Ollama Structured Outputs + `interrupt()` approval gate. 4. Temporal publishing workflow + LinkedIn (`/rest/posts`) and X (`/2/tweets` + chunked upload) adapters with read-after-write verification.	Local GPU with Ollama installed; developer apps registered on LinkedIn and X.	Proves the core hybrid architecture (LangGraph + Ollama + Temporal + Vault) end-to-end without waiting on multi-week Meta/Google audits.
M2: Sandboxed Ephemeral Workers & Hybrid Agentic RAG	1. Configure Docker `runsc` (gVisor) runtime, `sandbox_internal_net` (`internal: true`), Envoy forward proxy, and self-hosted `SearXNG`. 2. Build `SandboxManager` and `MCPToolGateway` with HMAC capability tokens. 3. Implement `pgvector` HNSW + `tsvector` hybrid store (`bge-m3` embeddings) and ephemeral Research/Competitor Workers (`Playwright` + `Trafilatura`).	M1 complete; Linux host kernel supporting `runsc`.	Eliminates arbitrary internet egress from workers and proves multi-worker fan-out without GPU VRAM thrashing.
M3: Media Generation Pipeline & Normalization Library	1. Build Media Worker integrations for Fal.ai and Replicate (Flux images, Kling/Luma video).	M1 & M2 complete; API keys for Fal.ai or Replicate.	Solves the "Local LAN vs. Public Media URL" blocker required by Meta, Pinterest,

Milestone	Scope & Engineering Deliverables	Technical Dependencies	Primary Risk Mitigated
	2. Implement `FFmpeg` / `Pillow` deterministic normalizer (JPEG conversion, H.264/AAC MP4 faststart, aspect-ratio validation) and `gemma4` vision QA check. 3. Configure signed HTTPS media staging ingress (Cloudflare Tunnel / R2 bucket with 15-min TTL).		and GBP APIs.
M4: Container & Video Platform Adapters (Instagram, Threads, Facebook, YouTube, Pinterest, GBP)	1. Implement Meta adapters (**Instagram** `/<IG_ID>/media_publish`, **Threads** `/threads_publish`, **Facebook Pages** `/feed` & `/videos`) with `content_publishing_limit` pre-checks. 2. Implement **YouTube Data API v3** resumable upload adapter (`308` recovery + 100/day bucket tracking + un-audited `private` guard). 3. Implement **Pinterest API v5** adapter (Trial sandbox detector) and **GBP API v4** `localPosts` adapter (0-QPM guard + phone/email regex sanitizer).	M3 public media staging complete; platform OAuth apps configured.	Isolates complex multi-step container polling, resumable binary streams, and platform-specific compliance guards.
M5: Unified Workspace UI (Next.js 15)	1. Build **Agent Monitoring** view with live SSE stream of LangGraph nodes, worker container lifecycles, and typed HITL approval forms. 2. Build **Content Calendar** (FullCalendar drag-and-drop synced to Temporal schedules), **Media Library**, **Campaign Manager**, and **RAG Knowledge Inspector**.	M1–M4 backend APIs stable.	Provides full operator visibility and ergonomic human-in-the-loop governance across all workflows.
M6: Closed-Loop Analytics, Anomaly Detection & Autonomous Optimization	1. Implement Temporal 6h/24h analytics ingestion activities across all 8 platforms into `post_metrics_timeseries`. 2. Implement Sunday `02:00 UTC` immutable Parquet/JSONL weekly backup workflow to MinIO. 3. Build cohort Z-score evaluator, `AnalyticsDiagnosticWorker`, `OptimizationProposal` HITL gate (`HITL-03`), and automatic retrospective indexing into `pgvector`.	M4 publishing & M2 RAG complete; historical or seeded post telemetry.	Closes the Observe–Analyze–Act–Evaluate feedback loop with statistical rigor and human safety gates.

11. Testing and Acceptance: Test strategy, Success Criteria, and Validation Procedures

11.1 Multi-Layer Testing Strategy

1. **Deterministic Unit & Schema Tests (`pytest`)**: Verify Pydantic `CampaignBlueprint` constraints (character limits, media aspect ratios, URN regexes), `AES-256-GCM` encryption/decryption roundtrips, idempotency hash generation, and GBP regex policy sanitization.
2. **Recorded HTTP Contract & Fault-Injection Tests (`vcrpy` / `respx`)**: Test all 8 platform adapters against recorded request/response fixtures covering: - LinkedIn `201 Created` (`x-restli-id`) and `429 TOO_MANY_REQUESTS` 24-hour cooldown handling. - YouTube resumable upload interrupted at 50% byte offset (`HTTP 503` followed by `Content-Range: bytes */LENGTH` → `HTTP 308 Resume Incomplete` recovery) and `HTTP 403 quotaExceeded` immediate stop. - Instagram and Threads container polling (`IN_PROGRESS` → `FINISHED` vs. `ERROR`) and `2207042` daily publishing cap enforcement. - X 3-step chunked upload (`initialize` → `append` → `finalize` → `processing_info.state == "succeeded"`). - Pinterest Trial Access sandbox flagging and GBP `0 QPM` (`429 RESOURCE_EXHAUSTED`) pre-flight detection.
3. **Container Sandbox Security Penetration Tests**: Automated test suite inside `worker-sandbox-image` running under `--runtime=runsc` verifying that (a) `uname -a` / `dmesg` confirms the gVisor application kernel, (b) root filesystem writes outside `/tmp` fail with `EROFS`, and (c) direct TCP/UDP sockets to `8.8.8.8` or external domains bypassing the Envoy proxy fail 100% of the time.
4. **Model & Agent Evaluation Harness**: Benchmark suite of 25 representative marketing directives evaluating Ollama (`qwen3:14b` and `qwen3:32b`) on (a) JSON schema compliance rate (target: `100%` over 50 runs at `temperature=0`), (b) tool selection accuracy without empty `tool_calls` regressions, and (c) zero hallucinated URLs in synthesized research briefs.

11.2 Milestone Acceptance Matrix (Six Mandatory Verification Gates per Milestone)

Milestone	1. Expected Behavior	2. Required Inputs & Outputs	3. Verification Procedure	4. Success Conditions	5. Failure Conditions	6. Evidence Required for Completion
M1: Vertical Slice	User prompt compiles to `campaign.json`, pauses at `interrupt()`, resumes on approval, and publishes a text+image post to LinkedIn and X via Temporal.	**In**: Prompt string, LinkedIn/X OAuth tokens. **Out**: Valid `campaign.json`, `scheduled_posts` row with `VERIFIED_PUBLISHED`.	Trigger workflow, kill FastAPI container during `interrupt()`, restart, send `POST /resume`, inspect Temporal UI and live LinkedIn/X profiles.	State recovers after restart; HTTP 201 returned; subsequent `GET` confirms live post URN/ID; replay with same `idempotency_key` creates 0 duplicates.	Unencrypted token in DB; duplicate post on retry; `tool_calls` empty when `format` set; status marked published without read-after-write `GET`.	PostgreSQL query dump of `scheduled_posts` showing `external_post_id`, `verified_at`, and HTTP 200 read-after-write verification log.
M2: Worker Sandbox & RAG	Master Orchestrator spawns 2 parallel `runsc` research workers, crawls allowlisted URLs via proxy, indexes findings in `pgvector`, and destroys containers.	**In**: Competitor research prompt, target domain list. **Out**: Cited strategy brief, `knowledge_chunks` rows, `worker_execution_audit` teardown logs.	Run research graph while capturing `docker events` and proxy logs; run egress bypass test from inside worker container.	Workers run under `runsc`, complete in `<60s`, are destroyed within `5s`, and hybrid RAG query retrieves cited chunks in `<150ms`.	Worker container remains running (`zombie`); direct `curl` bypassing proxy succeeds; Ollama OOM crash during parallel worker calls.	`docker events` log showing `create` → `die` → `destroy` under `runsc`, proxy access log, and `EXPLAIN ANALYZE` on `pgvector` HNSW query.
M3: Media Pipeline	Media Worker invokes Fal.ai/Replicate, downloads raw	**In**: `media_requirements` array from `campaign.json`. **Out**: Normalized MP4/JPEG	Execute media pipeline for a 4:5 Instagram image and 9:16 Reel video; run `ffprobe` and external `curl`	Image is baseline sRGB JPEG `≤5 MB`; video is H.264/AAC MP4 with `moov` atom	PNG passed to Instagram image endpoint; `moov` atom at end of MP4; staging URL unreachable	`ffprobe` JSON output, `media_assets` table row with `sha256_hash`, and external HTTP header

Milestone	1. Expected Behavior	2. Required Inputs & Outputs	3. Verification Procedure	4. Success Conditions	5. Failure Conditions	6. Evidence Required for Completion
	asset, normalizes via `FFmpeg`/`Pillow`, passes `gemma4` vision QA, and generates a 15-min signed HTTPS URL.	in MinIO, `media_assets` row, reachable HTTPS URL.	`-I` against the signed staging URL.	at front; external `curl` returns `HTTP 200` before TTL and `403/404` after expiry.	from external network or remains public permanently.	transcript verifying TTL expiration.
M4: 8-Platform Adapters	All 8 platform adapters execute their full upload, container polling, publishing, and read-after-write verification contracts (or deterministic sandbox/audit guards).	**In**: Approved multi-platform `campaign.json` + normalized media assets. **Out**: Verified platform post IDs/permalinks across all 8 adapters.	Run full contract suite (`respx` fault injection + live/sandbox credentials where available) including simulated mid-upload HTTP 503 drop on YouTube.	YouTube resumes via HTTP `308`; IG/Threads poll until `FINISHED`; Pinterest Trial flags `SANDBOX_ONLY`; GBP strips raw phone numbers and detects `0 QPM`.	Adapter marks post `VERIFIED_PUBLISHED` on container creation (`/<IG_ID>/media`) before `/media_publish` succeeds; infinite retry loop on YouTube `403 quotaExceeded`.	Automated contract test report (100% pass across all 8 adapters) + live/sandbox verification payloads stored in `scheduled_posts`.
M5: Unified Workspace	Operator manages campaigns, calendar slots, media assets, RAG documents, and HITL approvals with real-time SSE agent telemetry in Next.js UI.	**In**: Browser interactions (drag calendar slot, click Approve/Edit on interrupt). **Out**: Updated Temporal schedules and resumed LangGraph threads.	Execute Playwright E2E browser test covering campaign creation, SSE stream rendering, HITL copy edit, and calendar drag-and-drop reschedule.	UI renders live LangGraph node state within `500ms`; editing copy in HITL modal updates `campaign.json` state cleanly; calendar drag updates Temporal timer.	SSE stream drops without reconnection; UI displays "Published" before read-after-write verification completes.	Playwright E2E test execution report and HAR network trace confirming SSE and HITL resume payloads.
M6: Closed-Loop Analytics	System ingests cross-platform metrics, creates weekly Parquet backups, flags $Z < -1.5$ outliers, spawns diagnostic worker, and indexes approved learnings.	**In**: Live or seeded 14-day `post_metrics_timeseries` dataset ($N \geq 10$ posts). **Out**: Weekly MinIO Parquet backup, `OptimizationProposal` interrupt, new `knowledge_chunks` row.	Trigger analytics ingestion & weekly backup workflows; inject an underperforming cohort ($Z = -2.1$) and verify `OptimizationSubgraph` execution.	Weekly Parquet file written and readable via `duckdb`/`pandas`; outlier cohort triggers diagnostic worker and pauses at `HITL-03` with cited evidence.	Autonomous modification of live scheduled posts without `HITL-03` approval; outlier triggered on sample size $N < 5$.	MinIO backup manifest, `OptimizationProposal` JSON payload with statistical citations, and verified `pgvector` retrospective entry.

12. Risks and Open Decisions: Blockers, Unresolved Questions, Assumptions, and Required Decisions

12.1 High-Priority Technical and Operational Risk Register

Risk ID	Risk Description	Severity	Likelihood	Concrete Engineering Mitigation
RSK-01	**External Platform Audit & Verification Delays** (Meta Business Verification, YouTube Compliance Audit forcing `private` uploads, Pinterest Trial sandbox lock, GBP 60-day verified profile + `0 QPM` gate).	**Critical**	**Certain (100%)**	Initiate developer app registrations and compliance filings during Week 1 (parallel to Milestone 1); implement `AccountEligibilityState` guards in every adapter so un-audited platforms degrade gracefully to sandbox/private draft mode without crashing campaigns.
RSK-02	**Local GPU VRAM Starvation & Context Truncation** during concurrent orchestrator and worker execution on Ollama.	**High**	**High**	Enforce `OLLAMA_MAX_LOADED_MODELS=2` and `OLLAMA_NUM_PARALLEL=2`; standardize on one resident tool-calling model (`qwen3:14b` or `32b`) with explicit `num_ctx: 32768`; route deterministic tasks (transcoding, scheduling, metrics math) to Python/Temporal rather than LLMs.
RSK-03	**Ollama Empty `tool_calls` Regression** when combining `format` (JSON schema) and `tools` in a single request (Ollama Issue #8095).	**High**	**High**	Enforce an architectural invariant in `ollama_client.py`: raise a compile-time `ValueError` if a caller passes both `tools` and `format` in the same invocation; always separate ReAct tool-calling nodes from structured JSON extraction nodes.
RSK-04	**LangGraph Node Replay Side Effects** on `Command(resume=...)` (LangGraph Interrupt Rules).	**High**	**Medium**	Isolate `interrupt()` calls into dedicated lightweight gate nodes (`hitl_strategy_gate`, `hitl_campaign_gate`) containing zero external API calls or non-idempotent database `INSERT` statements prior to `interrupt()`.
RSK-05	**Pull-Based Media Fetch Failures** on Instagram, Threads, and GBP when self-hosting behind NAT.	**High**	**Medium**	Require a verified public HTTPS media staging endpoint (Cloudflare R2 presigned URL or Cloudflare Tunnel to a dedicated public-read MinIO staging prefix) tested via external reachability probe before calling `/<IG_ID>/media` or `localPosts`.

12.2 Coding-Agent Execution Strategy and Operating Rules

To allow a human engineer or autonomous coding agent to implement this specification without inventing ad-hoc architectural decisions, all implementation work must obey the following strict execution protocol:

1. **Stage Gate Enforcement**: Execute exactly one milestone (`M1` through `M6`) at a time. Never begin coding `M(n+1)` until all six verification gates for `M(n)` in Section 11.2 are satisfied with reproducible test output.
2. **Strict Separation of Cognitive vs. Deterministic Code**: Never place `time.sleep()`, cron loops, chunked HTTP binary uploads, or OAuth token refresh logic inside a LangGraph node or ephemeral worker. All time-delayed, retry-sensitive, or credential-bearing operations must be implemented as deterministic Temporal Activities inside `services/control-plane/app/workflows/`.
3. **Invariant Check on `ollama_client.py`**: Verify that `OllamaClient.chat_with_tools()` never sets `format`, and `OllamaClient.generate_structured()` always sets `format=schema.model_json_schema()`, `options={"temperature": 0, "num_ctx": 32768}`, and `tools=None`.
4. **Zero Plaintext Credentials in Logs or Worker Environments**: Never pass `social_credentials` rows into LangGraph state dictionaries (`OrchestratorState` is persisted in `checkpoints` JSONB) or worker container environment variables. Pass only opaque `credential_id` UUIDs and decrypt tokens strictly inside the isolated Temporal publishing activity at the moment of the HTTPS request.

13. Immediate Next Action: Executing Milestone 1 to Prove End-to-End Local Orchestration

The single highest-priority engineering action to execute immediately is **Milestone 1 (`M1`): Core Vertical Slice Initialization and External Platform Audit Filing**, executed in two parallel tracks:

1. **Track A — Administrative Unblocking (Day 1)**: Submit developer application registrations and verification requests for the four long-lead-time platforms: (a) file the Google Business Profile Basic API Access Form to lift the `0 QPM` default quota, (b) enable YouTube Data API v3 in Google Cloud Console and initiate OAuth Consent Screen Verification + the YouTube API Services Compliance Audit, (c) initiate Meta Business Verification for Instagram/Threads/Facebook Pages Advanced Access, and (d) apply for LinkedIn Community Management API access (`w_organization_social`, `r_organization_social`).
2. **Track B — Engineering Implementation (`M1` Vertical Slice)**: Initialize the `agentic-marketing-os` repository with `docker-compose.yml` (`postgres:16` with `pgvector`, `temporalio/auto-setup`, `minio/minio`, and `ollama/ollama`), implement the `AES-256-GCM` credential vault and Alembic migration for `brands`, `social_credentials`, `campaigns`, and `scheduled_posts`, implement the Master Orchestrator `StateGraph` (`ingest_directive` → `compile_campaign_blueprint` → `hitl_campaign_gate` → `dispatch_to_temporal`), and verify an end-to-end text+image post to LinkedIn (`/rest/posts`) and X (`/2/tweets`) with read-after-write confirmation.

14. Authoritative References and Verification Index

- **LinkedIn Posts API Official Reference (Version 202610)**: Microsoft Learn — LinkedIn Marketing Community Management Posts API
- **LinkedIn API Access Tiers and Permissions**: Microsoft Learn — Increasing Access to LinkedIn Marketing APIs
- **LinkedIn Rate Limits and Integration Analysis (2026)**: ConnectSafely — LinkedIn Post API Integration Guide 2026
- **YouTube Data API v3 Quota Model and Compliance Audits (2026)**: Outstand — YouTube API Pricing 2026: Quota, Upload Limits, Real Costs
- **YouTube Data API Official Quota Calculator**: Google for Developers — YouTube Data API Quota Calculator
- **Instagram Platform Content Publishing Reference (Updated Oct 20, 2026)**: Meta for Developers — Instagram Content Publishing Guide
- **Instagram API Rate Limits and Container Constraints (2026)**: KeyAPI — Instagram API Rate Limits 2026
- **Threads API Official Overview and Rate Limits**: Meta for Developers — Threads API Overview
- **Facebook Pages API Official Posts Reference (Updated Apr 17, 2026)**: Meta for Developers — Facebook Pages API Posts Guide
- **Meta App Review and Business Verification Walkthrough (2026)**: Meta for Developers — App Review Submission Guide
- **X (Twitter) API v2 Posting and Chunked Media Upload Guide (2026)**: Postproxy — X (Twitter) API Posting: 2026 Integration Guide
- **Cross-Platform Social Media API Rate Limits (2026)**: PostEverywhere — Social Media API Rate Limits Compared 2026
- **Pinterest API v5 Official Access Tiers and Sandbox Behavior**: Pinterest Developers — Understanding Access Tiers
- **Google Business Profile API Access, `localPosts`, and Quota Limits (2026)**: Breakreach — Google Business Profile API in 2026: Access, Posts, Limits
- **Google Business Profile Official Quota Documentation**: Google for Developers — Business Profile APIs Quota Limits
- **Postiz Open-Source Agentic Social Scheduler Repository**: GitHub — gitroomhq/postiz-app and AGPL-3.0 License
- **Mixpost Open-Source Social Media Management Repository**: GitHub — inovector/mixpost
- **Ollama Official Structured Outputs Documentation**: Ollama Documentation — Structured Outputs
- **Ollama Official Tool Calling Documentation**: Ollama Documentation — Tool Calling
- **Ollama Structured Output + Tool Calling Regression Tracker**: GitHub — ollama/ollama Issue #8095
- **LangGraph Official Persistence and Checkpointers**: LangChain Documentation — LangGraph Checkpointers
- **LangGraph Official Human-in-the-Loop Interrupts (Updated Oct 30, 2026)**: LangChain Documentation — LangGraph Interrupts
- **Temporal Durable Execution Architecture and Queue Comparison**: CodeNicely — Celery vs BullMQ vs Temporal Comparison (2026) and CoderCops — Temporal Durable Workflows in Production 2026
- **pgvector Official Repository and Hybrid Search Patterns**: GitHub — pgvector/pgvector and Crunchy Data — Hybrid Vector Search with Postgres and pgvector
- **gVisor (`runsc`) Container Sandbox Architecture**: gVisor Official Documentation and gVisor Docker Quick Start
- **External AI Video and Image Generation APIs**: Fal.ai Video APIs, Replicate Text-to-Video Collection, and Infrabase — Image and Video Generation APIs Compared (Oct 2026)